

Lập trình hướng đối tượng

Lớp và đối tượng trong Java

GV: ThS. Ngô Tiến Đức

Nội dung chính

- Lớp và đối tượng
- Tính đóng gói
- Tính kế thừa
- Tính trừu tượng
- Tính đa hình

Lớp và đối tượng

Nhắc lại về lớp và đối tượng:

- Lớp: Biểu diễn một tập hợp các đối tượng
- Đối tượng: Một thể hiện cụ thể (instance) của lớp
- Ví dụ: Bài toán quản lý bán ô tô của một cửa hàng ô tô
 - Lớp "Ô tô" chỉ chung tất cả các loại ô tô trong cửa hàng, lớp "Vinfast" chỉ chung tất cả các ô tô hãng Vinfast trong cửa hàng,...
 - Một đối tượng "Ô tô" chỉ một chiếc ô tô cụ thể, một đối tượng "Vinfast" chỉ một chiếc Vinfast cụ thể,...

Lớp và đối tượng

Nhắc lại về biến và phương thức:

- Biến: Lưu trữ dữ liệu
- Phương thức (hàm): Các câu lệnh thực hiện một số công việc (hành vi)
→ Một đối tượng cũng có thể coi là một biến trong đó bao gồm:
 - Các biến = thuộc tính (attributes)
 - Các phương thức = hành động đối tượng có thể thực hiện (operations)

Có thể coi lớp là một kiểu dữ liệu của đối tượng?

Lớp và đối tượng

Nhắc lại về kiểu dữ liệu:

- Kiểu dữ liệu nguyên thủy: `boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, `double`
- Kiểu dữ liệu không nguyên thủy/tham chiếu -> Kiểu "đối tượng". Ví dụ:
 - Lớp `String` biểu diễn các đối tượng có thể lưu trữ nhiều ký tự
 - Lớp `Scanner` biểu diễn các đối tượng có thể đọc luồng các ký tự dưới dạng token
 - ...

Lớp và đối tượng

Ví dụ lớp **Point** biểu diễn vị trí các điểm trong hệ tọa độ (số nguyên):

- Mỗi đối tượng Point có các thuộc tính:
 - Hoành độ: biến `x` kiểu `int`
 - Tung độ: biến `y` kiểu `int`
- Mỗi đối tượng Point có thể thực hiện các hành động:
 - Thay đổi tọa độ: Phương thức `setLocation (x, y)`
 - Tính khoảng cách đến một đối tượng Point khác (p): Phương thức `getDistance (p)`

Lớp và đối tượng

```
public class Point {  
    // attributes  
  
    int x;  
  
    int y;  
  
    // operations  
  
    public void setLocation(int x, int y) { ... }  
    public double getDistance(Point p) { ... }  
  
    ...  
  
    public static void main(String[] args) { ... }  
  
}
```

Lớp và đối tượng

- Khởi tạo một đối tượng mới: Sử dụng từ khóa `new`
 - Cú pháp: **<class> <name> = new <class> (<arguments>) ;**
 - VD: `Scanner sc = new Scanner(System.in) ;`
- Hàm khởi tạo cùng tên với lớp và không có kiểu trả về
- Nếu không viết hàm khởi tạo cho lớp, trình biên dịch tự hiểu hàm khởi tạo ở dạng mặc định không có tham số:

```
Point p1 = new Point() ;
```

```
// khởi tạo đối tượng p1 có tọa độ mặc định (0,0)
```

Lớp và đối tượng

- Một lớp có thể có nhiều hàm khởi tạo với các tham số khác nhau:

```
public class Point {  
    int x;  
    int y;  
    public Point() {}  
    public Point(int x) { ... }  
    public Point(int y) { ... } // problem?  
    public Point(int x, int y) { ... }  
    ...  
}
```

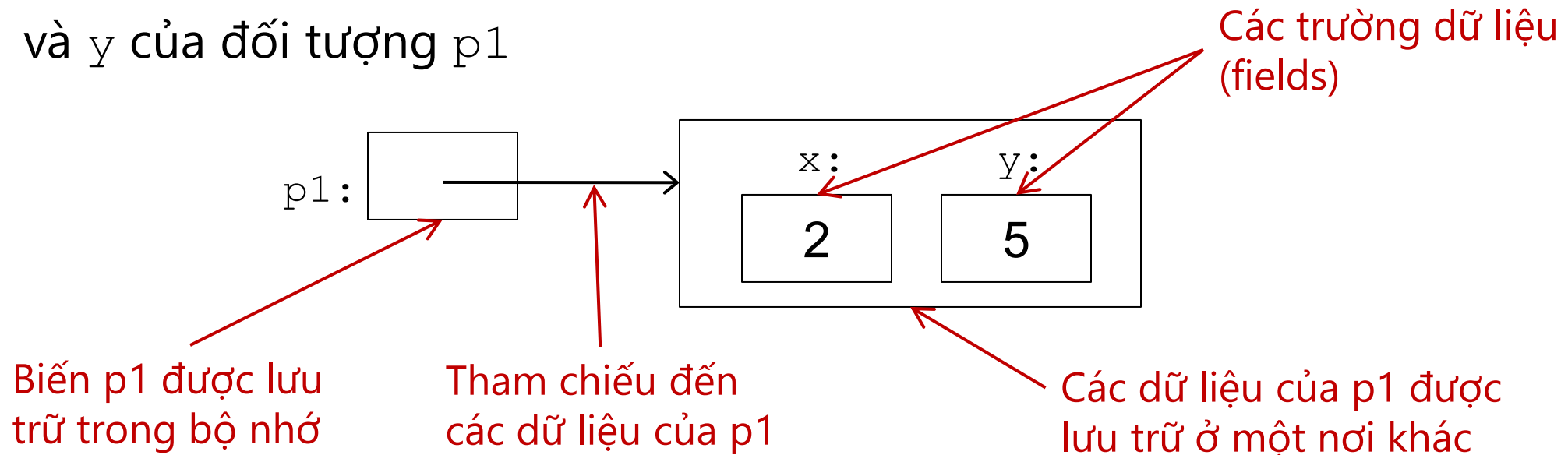
Lớp và đối tượng

- VD khởi tạo một đối tượng `p1` với tọa độ (2, 5):

```
public class Point {  
    ...  
    public static void main(String[] args) {  
        Point p0 = new Point();  
        Point p1 = new Point(2, 5);  
        ...  
    }  
}
```

Lớp và đối tượng

- Các biến đối tượng (object variables) sẽ tham chiếu tới vị trí trong bộ nhớ chứa dữ liệu của đối tượng đó
- VD: Biến `p1` lưu trữ địa chỉ của vị trí trong bộ nhớ chứa dữ liệu về giá trị `x` và `y` của đối tượng `p1`



Lớp và đối tượng

- **Dereferencing:** Sử dụng toán tử "." trên một đối tượng để truy cập vào các dữ liệu hoặc gọi các phương thức mà đối tượng tham chiếu tới
- Truy cập thuộc tính của đối tượng: **<variable> . <attribute>**
- Gọi phương thức của đối tượng: **<variable> . <method_name> (<params>)**

```
String s1 = "Hello world";  
System.out.println(s1.length());  
Point p0 = new Point();  
Point p1 = new Point(2, 5);  
System.out.println(p1.x);  
System.out.println(p1.getDistance(p0));
```

Lớp và đối tượng

- Một đối tượng có thể nhận giá trị `null`: Đối tượng không tham chiếu tới dữ liệu nào -> không có bộ nhớ cấp phát để lưu trữ dữ liệu
- Khi dereference một đối tượng `null` -> `NullPointerException`

```
Point p1 = null;  
System.out.println(p1.x) ;
```

```
Point p2 = null;  
p2.setLocation(2, 5) ;
```

Lớp và đối tượng

- Khi phương thức thành phần thao tác trên nhiều đối tượng: Cần sử dụng từ khóa **this**:
 - Cho phép truy cập vào đối tượng hiện tại
 - Cú pháp: **this.<attribute>**
 - Loại bỏ sự nhập nhằng giữa biến cục bộ, tham số và thuộc tính/dữ liệu của lớp khi có sự trùng tên
 - Không sử dụng bên trong các khối lệnh static

Lớp và đối tượng

```
public class Point {  
    int x;  
    int y;  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    ...  
}
```

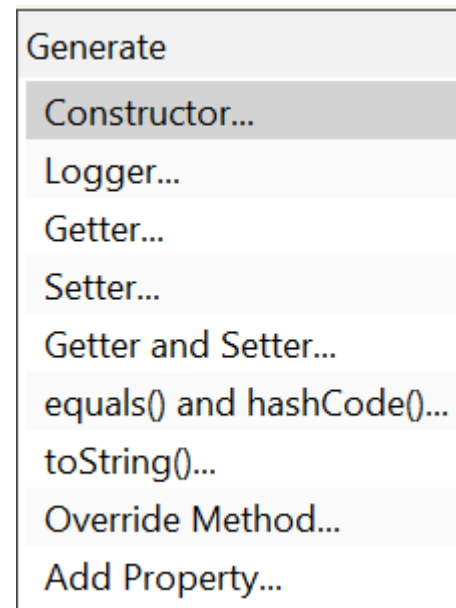
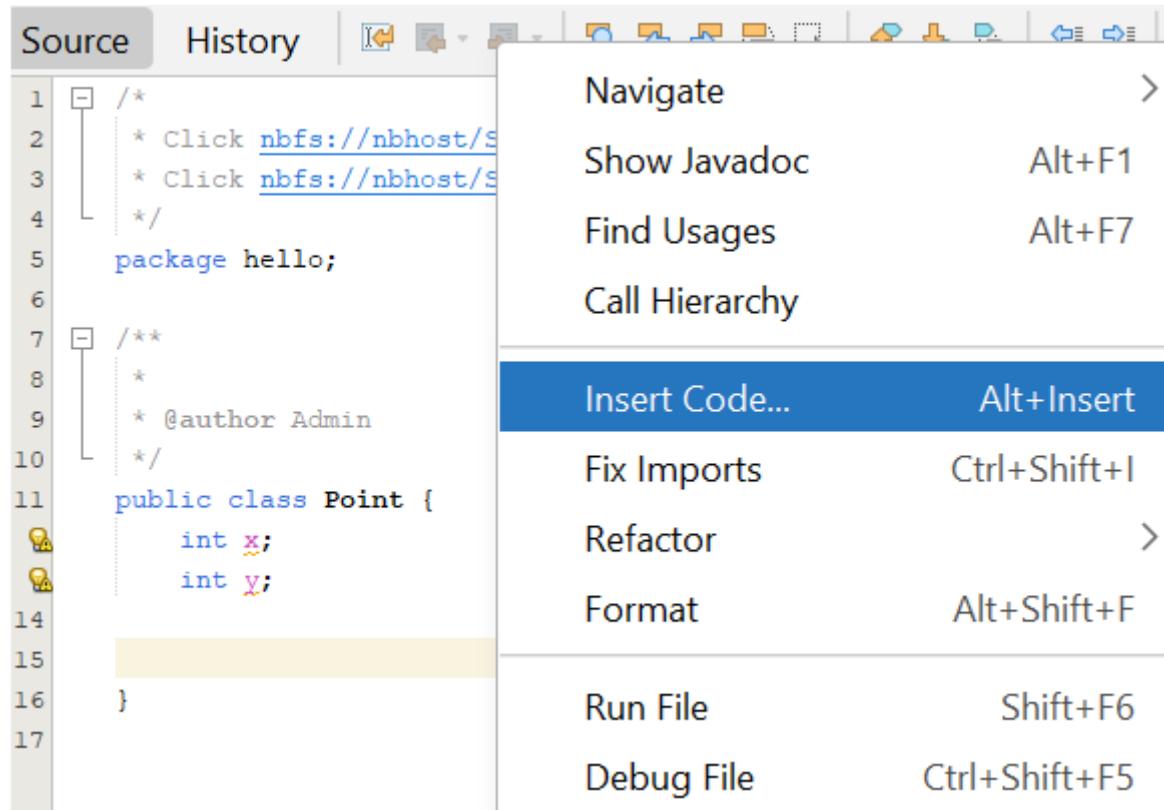
Lớp và đối tượng

- Tất cả các lớp đều có phương thức `toString()`: Chuyển đối tượng thành chuỗi ký tự
- Mặc định bao gồm Tên package, tên class và mã hex (base-16) hash-code
- VD: `System.out.println(p1.toString()); // Point@6acbcfc0`
- Có thể ghi đè (override) phương thức để in ra kết quả như mong muốn

```
@Override  
public String toString() {  
    return "x: " + x + ", y: " + y; // x: 2, y: 5  
}
```

Lớp và đối tượng

- IDE có thể hỗ trợ sinh một số phương thức cho lớp và đối tượng



Lớp và đối tượng

Luyện tập 1: Từ lớp **Point** trong ví dụ ở các slide trước

- Hoàn thiện phương thức `setLocation` và `getDistance`
- Bổ sung 2 phương thức `setX` và `setY` để cập nhật hoành độ và tung độ cho một điểm
- Tạo 2 đối tượng `Point` bằng hàm khởi tạo mặc định, sử dụng các phương thức `set` để gán tọa độ cho các điểm
- Tính khoảng cách giữa 2 điểm vừa tạo

Lớp và đối tượng

Luyện tập 2: Viết chương trình khai báo lớp sinh viên gồm các thông tin: mã sinh viên, tên, điểm

- Tạo 1 đối tượng sinh viên thứ nhất bằng hàm khởi tạo với các tham số là tất cả thuộc tính của lớp
- Tạo 1 đối tượng sinh viên thứ hai sử dụng hàm **set** cho từng thuộc tính (VD: `setMSV, setTen,...`)
- Tạo 1 đối tượng sinh viên thứ ba với các thông tin được nhập từ bàn phím
- In ra đầy đủ thông tin của các đối tượng, mỗi sinh viên trên một dòng

Tính đóng gói

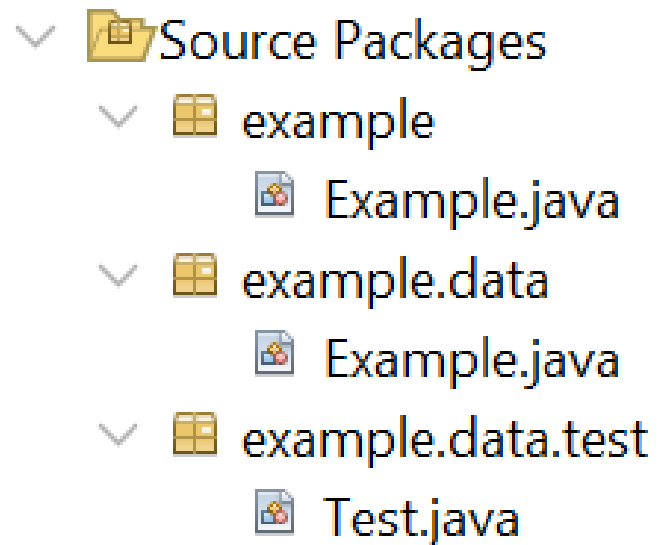
- Chương trình Java có thể được chia thành các gói (**package**) giống như các thư mục
- Một gói chứa tập hợp các lớp có liên quan tới nhau
- Tên gói: Tất cả là chữ thường
- Cú pháp khai báo một file nằm trong gói:

```
package <package_name>; // luôn đặt ở đầu file
public class <ClassName> {
    ...
}
```

Tính đóng gói

- Các gói có thể chứa các lớp cùng tên
- Trong gói có thể chứa các gói khác, khi đó tên gói sẽ phân cấp theo ký tự "."
- VD: File nằm trong gói có đường dẫn ***src > example > data > test***

```
package example.data.test;
```





- java.lang: Đã được import vào tất cả các lớp, có thể sử dụng không cần khai báo. VD: String, StringBuilder, các lớp bao (Wrapper classes),...
- java.util: Scanner, Arrays, Date,...
- java.io: File, FileReader, FileWriter,...
- ...

Tính đóng gói

Để sử dụng lớp nằm trong một gói nào đó:

- Cách 1: Khai báo đầy đủ bao gồm cả tên gói và tên lớp:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

- Cách 2: Import ở đầu file sau đó chỉ cần khai báo tên lớp không cần tên gói:

```
import <package_name>.<ClassName>; hoặc
```

```
import <package_name>.*;
```

```
...
```

```
Scanner sc = new Scanner(System.in);
```

- Không nên áp dụng cách 2 nếu trong file có 2 lớp cùng tên nằm trong 2 gói khác nhau

Tính đóng gói

VD bài toán quản lý nhân viên với lớp NhanVien

- Thuộc tính: `ten`, `ngaySinh`, `gioiTinh`, `phongBan`, `heSoLuong`
 - Phương thức: `tinhLuong`
 - Mỗi nhân viên có mức lương khác nhau nhưng tính chung theo công thức:
$$\text{Lương} = \text{Hệ số lương} * \text{Lương cơ bản} * \text{Tỉ lệ phần trăm}$$
 - Trong đó tỉ lệ phần trăm phụ thuộc vào phòng ban. VD phòng kế hoạch 100%, phòng nhân sự 110%,...
- > Tùy vào thuộc tính `phongBan` khác nhau mà phải dùng công thức với tỉ lệ khác nhau để tính lương cho mỗi nhân viên

Tính đóng gói

VD bài toán quản lý nhân viên với lớp NhanVien (tiếp)

- Cách tính lương là công việc bên trong phương thức `tinhLuong`
 - Cách gọi phương thức là giống nhau cho tất cả các đối tượng nhân viên ở bất cứ phòng ban nào
- > Với mỗi đối tượng nhân viên được khởi tạo ở bất cứ đâu bên ngoài lớp NhanVien, có thể tính được lương bằng cách gọi phương thức `tinhLuong` của lớp NhanVien và truyền các tham số cần thiết mà không (cần) biết chi tiết bên trong phương thức

Tính đóng gói

Tính đóng gói (**Encapsulation**): Sự giống nhau về cách sử dụng phương thức cho các đối tượng của cùng một lớp, mặc dù bên trong phương thức có các cách tính toán khác nhau với các đối tượng khác nhau

- Che giấu thông tin: Cho phép các đối tượng khác gọi và sử dụng phương thức nhưng ẩn đi cài đặt chi tiết
- Tăng khả năng tái sử dụng và hạn chế việc sửa mã nguồn: Khi phải thay đổi phương thức (VD: công thức tính toán), chỉ cần thay đổi bên trong chương trình chứa phương thức, không cần thay đổi các chương trình gọi đến phương thức

Tính đóng gói

Chỉ định/Phạm vi truy cập (**Access modifier**):

- public: Có thể được truy cập từ bất cứ đâu
- protected: Có thể truy cập trong phạm vi lớp và từ những lớp con/lớp kế thừa (subclasses)
- private: Chỉ có thể truy cập trong phạm vi lớp
- Mặc định - Không khai báo chỉ định (no modifier): Có thể truy cập từ các thành phần trong cùng gói

Tính đóng gói

Modifier	Class	Package	Subclass	World
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<code>no modifier*</code>	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

Bảng tóm tắt phạm vi truy cập của các modifier

Tính đóng gói

VD file Point.java

```
public class Point {  
    int x;  
    private int y;  
}
```

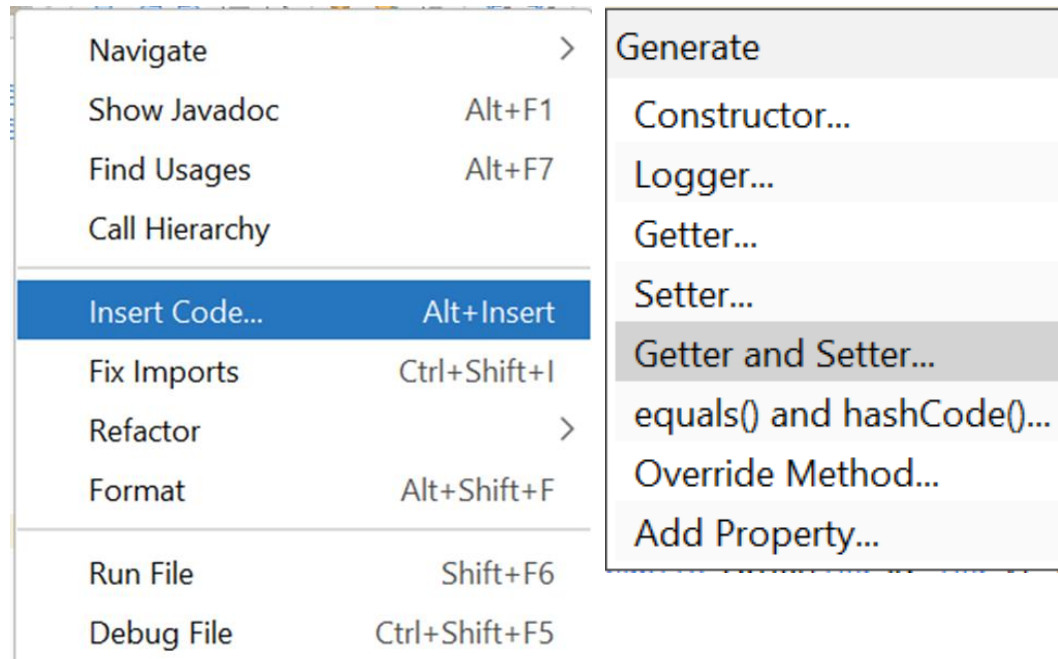
Một file khác trong cùng gói gọi đến lớp Point:

```
Point p = new Point(1, 2);  
System.out.println(p.x); // ok  
System.out.println(p.y); // lỗi private access
```

Tính đóng gói

Các phương thức **getter** (accessor) và **setter** (mutator):

- getter : Cho phép truy xuất thuộc tính private của đối tượng (read-only)
- setter: Cho phép thay đổi giá trị thuộc tính private của đối tượng



VD lớp Point:

```
public class Point {  
    private int x;  
  
    ...  
  
    public int getX() {  
        return x;  
    } // getter  
  
    public void setX(int x) {  
        this.x = x;  
    } // setter  
  
    ...  
  
}
```

Tính đóng gói

VD một file khác trong cùng gói gọi đến lớp Point:

```
Point p = new Point(1, 2);
```

```
System.out.println(p.x); // lỗi private access
```

```
System.out.println(p.getX()); // ok
```

```
p.x = 2 // lỗi private access
```

```
p.setX(2); // ok
```

- Tại sao phải dùng getter và setter mà không để modifier của thuộc tính là public cho nhanh?

Luyện tập

Luyện tập: Tạo một file Main.java, viết chương trình trong Main truy cập đến các thuộc tính từ lớp Point (hoặc lớp Student) vừa viết, thử thay đổi phạm vi truy cập của các thuộc tính trong Point để thấy sự khác nhau. Sau đó thử các hàm getter và setter.

Áp dụng cho 2 trường hợp:

- File Main.java nằm cùng gói với file Point.java
- File Main.java nằm ở gói khác

Tính kế thừa

Ví dụ xét bài toán quản lý nhân sự một trường Đại học, trong đó có các lớp quản lý thông tin nhân viên và sinh viên:

<i>Lớp Nhân viên</i>	<i>Lớp Sinh viên</i>
Thuộc tính: Tên Ngày sinh Giới tính Lương	Thuộc tính: Tên Ngày sinh Giới tính Lớp
Phương thức: Nhập/xem tên Nhập/xem ngày sinh Nhập/xem giới tính Nhập/xem lương	Phương thức: Nhập/xem tên Nhập/xem ngày sinh Nhập/xem giới tính Nhập/xem lớp

Tính kế thừa

Theo cách thông thường:

- Khai báo trùng nhau hai lần cho các thuộc tính: tên, ngày sinh, giới tính
 - Viết code trùng nhau hai lần các phương thức: nhập/xem tên, nhập/xem ngày sinh, nhập/xem giới tính
 - Nếu khi có sự thay đổi về kiểu dữ liệu (VD: ngày sinh) phải sửa lại chương trình hai lần tại hai nơi
- > Cần sử dụng kế thừa: nhóm các phần giống nhau của các lớp thành một lớp chung, sau đó cho các lớp ban đầu kế thừa lại lớp mới được tạo ra

Tính kế thừa

Ví dụ tạo ra lớp Người chứa các thuộc tính và phương thức chung, sau đó cho 2 lớp Nhân viên và Sinh viên kế thừa từ lớp Người

<i>Lớp Người</i> Thuộc tính: Tên Ngày sinh Giới tính Phương thức: Nhập/xem tên Nhập/xem ngày sinh Nhập/xem giới tính	
<i>Lớp Nhân viên kế thừa từ lớp Người</i> Thuộc tính: Lương Phương thức: Nhập/xem lương	<i>Lớp Sinh viên kế thừa từ lớp Người</i> Thuộc tính: Lớp Phương thức: Nhập/xem lớp

Tính kế thừa

Tính kế thừa (**Inheritance**): Phát triển các lớp mới dựa trên lớp đã có -> tránh sự trùng lặp code -> tính tái sử dụng

- Các lớp có chứa thuộc tính và phương thức chung gọi là lớp cha (parent/superclass) hay lớp cơ sở (base class)
- Các lớp kế thừa từ lớp cha gọi là lớp con (child/subclass) hay lớp dẫn xuất (derived class)
- Một lớp cha có thể có nhiều lớp con
- Một lớp con chỉ có thể kế thừa một lớp cha

- Cú pháp: Sử dụng từ khóa `extends`

```
class Person {  
    protected String name;  
    protected String dob;  
    ...  
}  
  
class Employee extends Person {  
    private int salary;  
    ...  
}
```

Tính kế thừa

- Quan hệ giữa lớp con với lớp cha được gọi là **is-a** (là một) và không có ngược lại. VD: Nhân viên **là một** Người, ~~Người là một Nhân viên~~
- Lớp con có tất cả các thuộc tính và phương thức của lớp cha (với phạm vi truy cập thích hợp)
- Lớp con có thể có các thuộc tính và phương thức riêng. VD: thuộc tính lương, phương thức nhập/xem lương
- Lớp con có thể ghi đè (override) các phương thức kế thừa từ lớp cha. VD: thay đổi phương thức nhập/xem tên

Tính kế thừa

Trường hợp lớp cha và lớp con không cùng thuộc một gói, lớp con sẽ không thể kế thừa các thành phần có phạm vi truy cập mặc định từ lớp cha

Modifier	Class	Package	Subclass	World
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<code>no modifier*</code>	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

Tính kế thừa

- Ví dụ ghi đè

```
class Person {  
    String name;  
    public void printName() {  
        System.out.println("name: " + name); // this.name  
    }  
    ...  
}
```

- Ví dụ ghi đè

```
class Employee extends Person {  
    @Override  
    public void printName() {  
        System.out.println("employee name: " + name);  
    }  
    ...  
} // nằm cùng package với class Person
```

Tính kế thừa

Khi khởi tạo đối tượng thuộc lớp con: Phương thức khởi tạo của lớp con luôn gọi phương thức khởi tạo của lớp cha ở câu lệnh đầu tiên

- Nếu lớp cha có phương thức khởi tạo không phải mặc định: Lớp con cần bổ sung hàm khởi tạo với từ khóa **super**: `super (<params>) ;`
- Cũng có thể dùng **super** để gọi các phương thức của lớp cha:
`super.<method_name> (<params>) ;`

Tính kế thừa

```
class Person {  
    String name;  
    String dob;  
    public Person() { }  
    public Person(String name, String dob) {  
        this.name = name;  
        this.dob = dob;  
    }  
    public void printName() {  
        System.out.println("name: " + name); // this.name  
    }  
}
```

Tính kế thừa

```
class Employee extends Person {  
    public Employee() {  
        super(); // có thể có hoặc không  
    }  
    public Employee(String name, String dob) {  
        super(name, dob);  
    }  
    @Override  
    public void printName() {  
        super.printName();  
        System.out.println("employee name: " + name);  
    }  
}
```

Luyện tập

Luyện tập: Viết chương trình cài đặt các lớp Employee và Student kế thừa từ lớp Person, trong đó file Employee.java nằm cùng gói với Person.java, còn Student.java nằm ở gói khác.

Thử lần lượt các trường hợp phạm vi truy cập của các thuộc tính và phương thức trong Person là private, protected và không có phạm vi truy cập. Sau đó truy cập đến các thuộc tính và phương thức của lớp Person từ các lớp con Employee và Student.

Tính trừu tượng

- Tính trừu tượng (**Abstraction**): Ẩn đi cài đặt chi tiết và chỉ hiển thị tính năng (tên các phương thức) với người dùng
- Ví dụ: Các lớp Square (hình vuông), Circle (hình tròn) và Rectangle (hình tam giác) kế thừa từ lớp Shape (Hình), mỗi hình có một công thức tính diện tích khác nhau -> tại lớp Shape chỉ khai báo tên phương thức tính diện tích `calculateArea` mà không có cài đặt, việc cài đặt sẽ được viết riêng trong từng lớp con

Tính trừu tượng

Lớp trừu tượng (**Abstract class**):

- Từ khóa: **abstract** đứng trước `class` khi khai báo lớp
- Có thể chứa các phương thức trừu tượng: Các phương thức không có phần cài đặt, cũng bao gồm từ khóa **abstract** khi khai báo
- Các lớp con kế thừa từ lớp trừu tượng bắt buộc phải cài đặt các phương thức abstract của lớp cha
 - Nếu không các lớp kế thừa cũng phải là lớp trừu tượng
- Không thể khởi tạo đối tượng từ lớp trừu tượng (khai báo thì được)



49

Tính trừu tượng

Interface: Trừu tượng hoàn toàn

- Từ khóa: **interface** thay cho `class` khi khai báo
- Các biến trong interface (nếu có) phải là `static` và `final`
- Tất cả các phương thức phải ở dạng trừu tượng
 - Khai báo không cần từ khóa **abstract** và không thể là `final`
- Các lớp kế thừa từ interface: sử dụng từ khóa **implements**
- Các lớp con kế thừa từ interface bắt buộc phải cài đặt tất cả các phương thức trong interface, nếu không thì lớp phải là trừu tượng



51

Tính trừu tượng

Một số lưu ý khi sử dụng interface:

- Một lớp không thể kế thừa từ nhiều lớp nhưng có thể kế thừa nhiều interface -> đa kế thừa trong Java
- Lớp trừu tượng có thể kế thừa interface với từ khóa **implements**
- Interface có thể kế thừa các interface khác với từ khóa **extends**
- Phạm vi truy cập của các thành phần trong interface mặc định là public

Vậy abstract class và interface khác gì nhau?

Tính đa hình

- Tính đa hình (**Polymorphism**): Khả năng sử dụng cùng một mã với nhiều loại đối tượng khác nhau và hoạt động khác nhau hay khả năng một đối tượng có thể thực hiện một tác vụ theo nhiều cách khác nhau
- Đã học: Tính đa hình với các phương thức
 - Nạp chồng (Overloading): Cho phép một lớp có nhiều phương thức cùng tên nhưng với các tham số khác nhau
 - Ghi đè (Overriding): Cho phép lớp con định nghĩa lại các phương thức kế thừa từ lớp cha

Tính đa hình

Đa hình với các đối tượng:

- Upcasting: Biến được khai báo kiểu lớp cha tham chiếu tới đối tượng của lớp con, có thể thực hiện tự động

- Ví dụ upcasting với lớp Employee là lớp con của Person

```
Person p = new Employee();
```

- Khi này `p` chỉ có thể gọi các phương thức ở lớp Person
- Nếu phương thức được ghi đè ở lớp con thì thực thi cài đặt bên lớp con

Tính đa hình

Đa hình với các đối tượng:

- Downcasting: Ngược lại với upcasting, phải khai báo kiểu tường minh
- Ví dụ downcasting với lớp Employee là lớp con của Person

```
Employee e = new Person(); // lỗi
```

```
Employee e = (Employee) new Person(); // ok
```

```
Person p = new Employee(); // upcasting
```

```
Employee e = p; // lỗi
```

```
Employee e = (Employee) p; // ok
```

Tính đa hình

Trong quá trình downcasting có thể gặp ngoại lệ `ClassCastException`

-> toán tử `instanceof` kiểm tra một đối tượng có thuộc một lớp không

```
Person p1 = new Employee();  
System.out.println(p1 instanceof Employee); // true  
Employee e1 = (Employee) p1; // ok  
Person p2 = new Person();  
System.out.println(p2 instanceof Employee); // false  
Employee e2 = (Employee) p2; // ClassCastException
```

Luyện tập

Luyện tập: Viết chương trình cài đặt đầy đủ cho các lớp Person, Employee và Student với các trường hợp:

- Person là lớp bình thường
- Person là lớp trừu tượng
- Person là interface

Thử khởi tạo đối tượng Person cho từng trường hợp.

Khởi tạo các đối tượng Employee và Student, in ra lương của nhân viên và lớp của sinh viên.

Vào ra file với đối tượng

- Sử dụng `ObjectInputStream` và `ObjectOutputStream`
- Đối tượng đọc/ghi phải là `Serializable`: Class của đối tượng kế thừa từ interface `Serializable`

```
import java.io.Serializable;
```

```
public class Student implements Serializable {  
    private String name;  
    private int dob;  
    ...  
}
```

Vào ra file với đối tượng

```
// ghi ra file  
Student student = new Student("Nguyen Van A", 20);  
FileOutputStream fos = new FileOutputStream("student.dat");  
ObjectOutputStream oos = new ObjectOutputStream(fos);  
oos.writeObject(student);  
  
// đọc từ file  
FileInputStream fis = new FileInputStream("student.dat");  
ObjectInputStream ois = new ObjectInputStream(fis);  
Student student = (Student) ois.readObject();
```



- 60